Ailixir Intelligence: Software Architecture

Allixir Intelligence is an AI-powered mobile application that transforms unstructured documents into structured, actionable knowledge. Users can upload or scan documents (e.g., medical reports, invoices), from which the system extracts domain-specific, meaningful data fields and organises them into a personalised knowledge base. A voice/chat interface enables users to ask precise, context-aware questions. Responses are generated using retrieval-augmented generation (RAG) over the user's personal knowledge base, with optional enrichment from external knowledge sources such as research papers.

Technology Stack

Frontend

- **React Native:** Framework for native apps using React and TypeScript
- **Expo:** a React Native platform that provides ready-to-use APIs such as camera functionality
- **Tamagui:** cross-plattform UI kit and style compiler that optimizes performance by flattening component trees
- **TanStack Query:** data fetching with caching, invalidation and request deduping
- **Jotai:** Atom-based state management library

Database Tier

- **Cloud Firestore:** User-specific data storage of the frontend app (e.g. chat history, ...)
- **Vector DB:** Storage of backend data (e.g. from RAG, extracted data from OCR, knowledge graphs, ...). Options include **MongoDB** and **Vertex AI Feature Store**

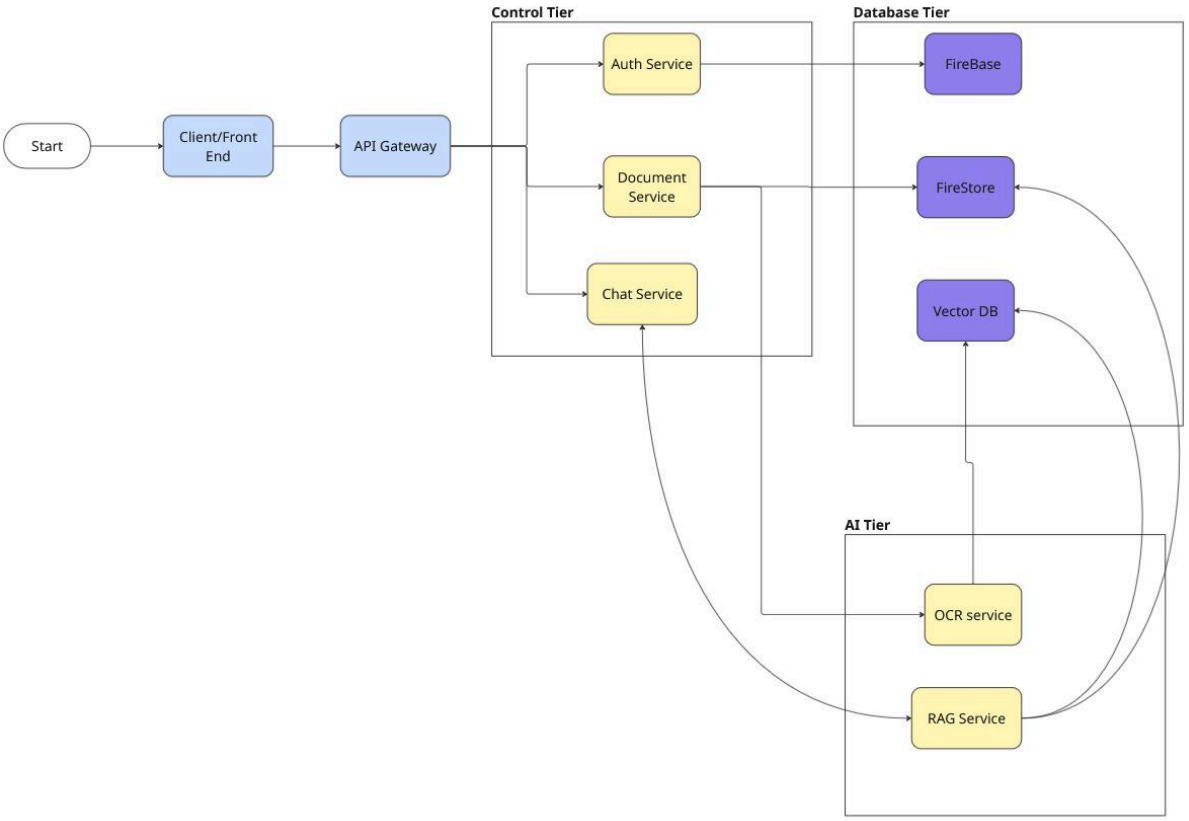
AI Tier

- **OCR models:** Data extraction from documents with **Google Document AI** or open source options such as **PaddleOCR** and **QwenOCR**
- **RAG Service: LangChain/LangGraph** for context-aware conversation handling
    - **Knowledge Graphs:** Implemented using **LangGraph**. In case of complex relations we may use **Neo4j** or **Graphiti**

Control Tier

- **Firebase:** User authentication for the frontend app, as per recommendations of the industry partner
- **Google Cloud Run:** Backend services can be run and will be exposed using an API gateway

Runtime Components



Code Components

The diagram describes the **static structure** of the Ailixir Intelligence source tree - how the code is packaged, where each piece lives in the monorepo, and how those packages depend on one another at compile time. It is a **build-time view**, not a runtime view: there are no requests, queues, or databases drawn here. **Solid arrows** mean "this artifact generates that one." **Dashed arrows** mean "this package imports that one."

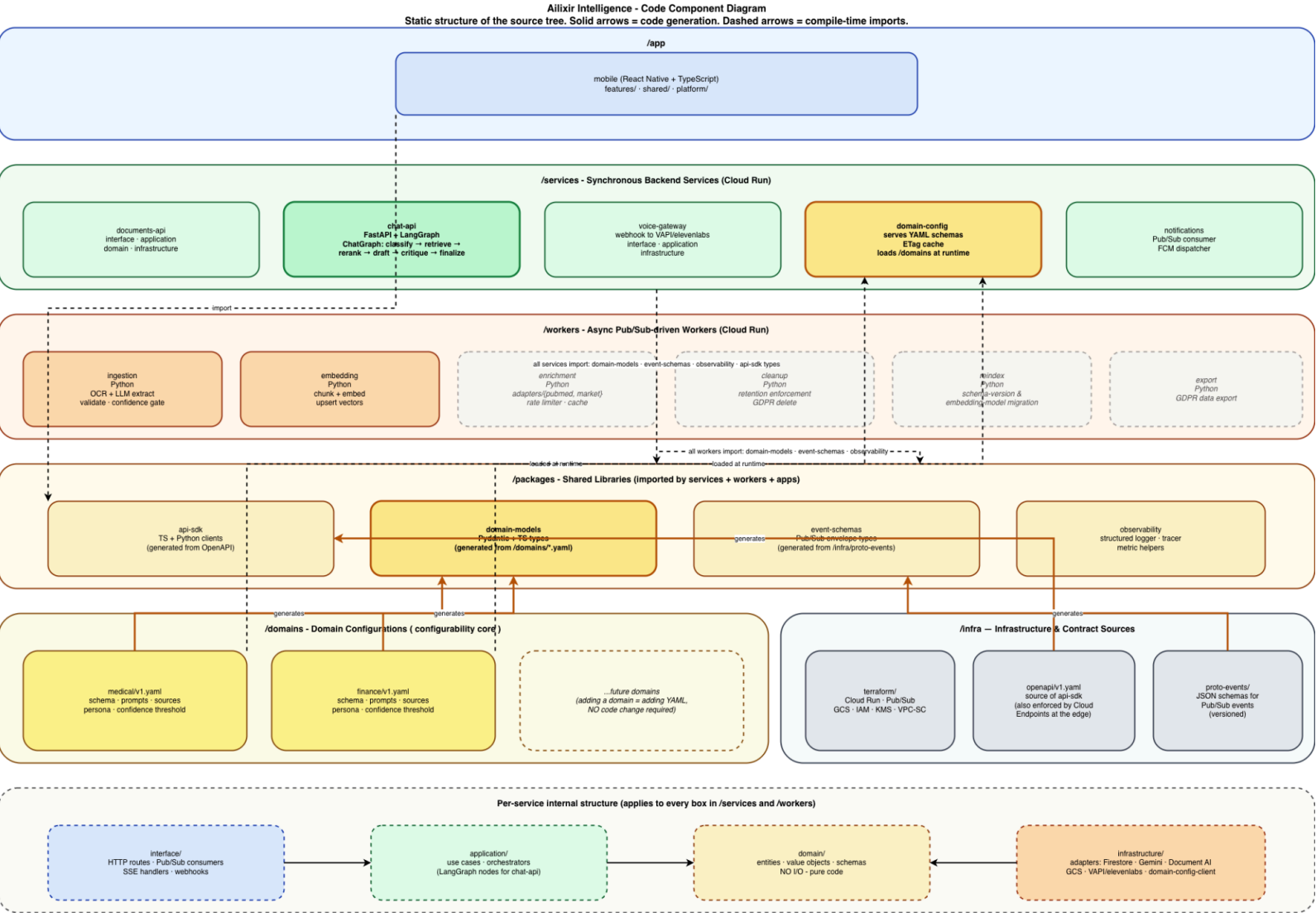
The diagram is also uploaded as a pdf and can be magnified in our github (for better readability).

The five top-level folders

The repository is divided into five top-level folders, each with a single responsibility:

- **`/app`** - the React Native + TypeScript mobile client. It is the only user-facing surface; a web client is explicitly out of scope. Internally it follows a `features/` · `shared/` · `platform/` split so individual screens can be added or swapped without touching cross-cutting code.

- **`/services`** - synchronous, request/response Cloud Run containers that the mobile app calls directly: `documents-api`, `chat-api` (FastAPI + LangGraph), `voice-gateway`, `domain-config`, and `notifications`. These services are tuned for low latency.



- **`/workers`** - asynchronous, Pub/Sub-driven Cloud Run containers: `ingestion`, `embedding`, `enrichment`, `cleanup`, `reindex`, and `export`. They handle slow, bursty, or scheduled work that must never block a user request - OCR, LLM extraction, vector upserts, retention enforcement, GDPR tasks.
- **`/packages`** - shared libraries imported by every service, worker, and app: `api-sdk` (typed HTTP client generated from OpenAPI), `domain-models` (Pydantic + TS types generated from `/domains/*.yaml`), `event-schemas` (Pub/Sub envelope types generated from `/infra/proto-events`), and `observability` (logger, tracer, metric helpers).
- **`/domains` + `/infra`** - the **contract sources**. `/domains/medical/v1.yaml` and `/domains/finance/v1.yaml` declare each domain's schema, prompts, allowed sources, persona, and confidence threshold. `/infra` holds Terraform, the OpenAPI specification, and the JSON schemas for Pub/Sub events. These two folders are the single source of truth for everything that is generated.

How the configurability requirement is satisfied

The defining requirement of the project is that switching between medical, finance, or any future domain must happen **without code changes**. The diagram makes this concrete: the YAML files in `/domains` are inputs to the build (they generate the `domain-models` package) and inputs to runtime (they are loaded by the `domain-config` service, which other services pull via an ETag-cached client). Adding `legal/v1.yaml` therefore reaches every service without any service knowing a new domain was added. This is the load-bearing property of the architecture.

How the contract sources prevent drift

Three things are **generated**, not hand-written:

1. `openapi/v1.yaml` → `api-sdk`. The mobile app and the services call each other through a typed client; the same OpenAPI document is enforced by Cloud Endpoints at the edge, so requests that do not match the schema cannot enter the system.
2. `/domains/*.yaml` → `domain-models`. Backend Python and the React Native app use the same field names, the same enums, and the same shapes - defined in one place.
3. `/infra/proto-events/*` → `event-schemas`. A producer and a consumer of the same Pub/Sub topic cannot disagree on the envelope, because both import types generated from the same JSON schema.

This eliminates an entire class of bugs - the type drift between layers - before the code ever runs.

Per-service internal structure (hexagonal)

Every box inside `/services` and `/workers` is internally split into the same four folders, shown in the bottom band of the diagram:

- `interface/` - HTTP routes, Pub/Sub consumers, SSE handlers, webhooks. The thin layer that translates external protocols into method calls.
- `application/` - use cases and orchestrators. For `chat-api` this is where the LangGraph nodes live (`classify → retrieve → rerank → draft → critique → finalize`).
- `domain/` - entities, value objects, schemas. Pure code with no I/O. This is the part that must remain unaware of Firestore, Gemini, or HTTP.
- `infrastructure/` - adapters: Firestore, Gemini, Document AI, GCS, VAPI/ElevenLabs, the `domain-config` client. Anything that talks to the outside world.

Dependencies always point **inward** toward `domain/`. This keeps business rules testable without spinning up Google Cloud and makes adapters swappable - e.g. replacing Document AI with another OCR provider is a one-folder change, not a system-wide refactor.

Key design choices, in one line each

- **Sync/async split.** User-facing services stay fast; expensive work runs in workers behind Pub/Sub, which gives retries and backpressure for free.
- **Contract-first generation.** Three generators (OpenAPI, domain YAML, proto-events) eliminate type drift between mobile, services, and workers.
- **Domain configuration as data, not code.** Adding a new domain means adding one YAML file — not opening a pull request against every service.
- **Hexagonal per service.** Adapters live at the edges, pure domain logic at the center; the system is portable and testable in isolation.